# 1001 最大的公约数

## 题目描述

给定一个正整数 $n$，求两个正整数 $a, b$ 满足 $a + b = n$ 且 $a \neq b$ 时，$\max(\gcd(a, b))$ 的值。

## 解题思路

### 1. 核心推导

由于 $a$ 和 $b$ 在题目中地位平等，且题目要求 $a \neq b$，我们可以不妨设 $a > b$。

利用 **欧几里得算法（辗转相除法）** 的性质：

$$\gcd(a, b) = \gcd(a, n - a) = \gcd(a, n)$$

**结论 1：** 题目所求的 $\gcd(a, b)$ 本质上一定是 $n$ **的某个约数**。设该约数为 $g$。

为了使 $g$ 最大，我们需要考察 $n$ 的约数性质：

- **排除 $n$ 本身**：因为 $b \geq 1$，所以 $a < n$，故 $\gcd(a, n) \neq n$。
- **排除 $n/2$**：如果最大公约数是 $n/2$，则 $a$ 必须是 $n/2$ 的倍数。由于 $n/2 < a < n$，这是不可能的；除非 $a = n/2$，但这会导致 $b = n/2$，即 $a = b$，与题目条件冲突。

**结论 2：**
题目等价于寻找 $n$ **的最大真约数 $g$，且满足 $g \neq n/2$**。

### 2. 算法转化

我们要找一个最大的因子 $g < n/2$。
根据 $n = k \times g$，这等价于找一个**最小的因子 $k$，使得 $n/k < n/2$**。
化简不等式得：$k > 2$。
因为 $k$ 是整数，所以条件转化为：**寻找 $n$ 的最小约数 $k$，满足 $k \geq 3$**。

## 3. 算法流程

1. **初始化保底**：
   - 若 $n$ 为偶数（且 $n \neq 4$），即使找不到其他因子，$n$ 至少包含因子 2。虽然 $n/2$ (对应 $k=2$) 不能取，但对于形如 $2 \times p$ 的数，除去 $p$ 和 $2p$ 外，剩下的最大因子通常是 2（对应拆分为 $2+(n-2)$）。所以偶数保底 `ans = 2`。
   - 若 $n$ 为奇数或 $n=4$，保底 `ans = 1`。
2. **贪心查找**：
   - 从小到大枚举 $i$（从 3 开始）。
   - 找到的第一个能整除 $n$ 的 $i$，即为满足 $i \geq 3$ 的**最小因子**。
   - 此时 $n/i$ 即为满足条件的**最大约数**。
   - 找到后立即输出并退出循环。

# 4.易错点总结

### 1.忽略 $a \neq b$ 的限制：

- **典型错误**：输入 6，输出 3。
- **分析**：$6$ 的最大真约数是 $3$，但取 $3$ 意味着 $a=3, b=3$，违规。正确答案是取 $6$ 的次大因子 $2$（对应 $2+4=6$）。
- **代码处理**：循环从 i=3 开始，天然避开了 i=2 (即 $n/2$) 的非法情况。

### 2.偶数的保底遗漏：

- **典型错误**：输入 10，输出 1。
- **分析**：$10 = 2 \times 5$。循环从 3 开始枚举到 $\sqrt{10} \approx 3$，找不到因子。如果默认 ans=1 就会出错。
- 实际上 $10$ 可以拆成 $2+8$，$\gcd = 2$。所以偶数必须特判保底为 2。

### 3.特殊数字 $n=4$：

- $4$ 是偶数，但不能取 $n/2 = 2$。
- $4$ 没有 $\geq 3$ 的真因子。
- 所以 $4$ 的答案只能是 1。代码中 n != 4 的特判必不可少。

# 5.复杂度分析

- **时间复杂度**：最坏情况下（$n$ 为大质数）循环运行 $\sqrt{n}$ 次。总复杂度 $O(T\sqrt{n})$。
  对于 $T=10^4, n=10^9$，总运算量约 $3 \times 10^8$，C++ 可在 1 秒内通过。

- **空间复杂度**：$O(1)$。

# 1002 雪蜜冰城柠檬茶的神秘成分魔法

## 题面

有一个数组，支持三种操作：把所有 `x` 变成 `y` 、在末尾 append `x` 、删除所有 `x` ，执行完输出最终数组（保持原出现顺序，删除的跳过）。

## 题解

发现删除实际上可以看成变成0，最后0不输出即可。把初始数组当作若干个“append”，那么实际上初始为空，只需维护两种操作：加数和替换。

正向替换会影响后来添加的元素和后续替换；把事件倒过来，替换就变成更新一个值到最终值的映射，追加就直接查映射得到最终值。因此考虑 **正难则反**：把整个事件序列反过来做。所有事件倒序处理——这样“把 x 改为 y”变成把 `x` 指向 `y` 的当前最终值；“append”变成读取那个位置最终值。

**需要维护的东西**：一个从值到“当前代表值”的映射 `f` （只记录出现过的键），以及答案列表 `ans` （倒序收集 append 的最终值，最后再反转输出）。

**具体操作**：

- 正向 `2 x` （append x） → 反向遇到它：读 `f[x]` （若 f[x] 未设置就认为是 x），把读到的值放入 `ans` （先存后反转）。
- 正向 `1 x y` （把 x 改成 y） → 反向：设置 `f[x] = f[y]` （用当前 f[y] 的值，保证传递性）。如果 y 未在映射里就用 y 本身。
- 正向 `3 x` （删除所有 x） → 反向：把 `f[x] = 0` ，表示这些位置最终被删掉；在输出阶段跳过 0。

# 1003 杂草蔓延

注意到，每蔓延一次，杂草块的图形周长单调不增。我们知道最后图形的周长为 $(n+m)*2$，故初始时刻的图形周长 $C \geq (n+m)*2$ 。考虑若初始情况下杂草块两两不相连，则杂草块的数量的个数为

$\frac{n+m}{2}$ ，即初始情况杂草块数 $num \geq \frac{n+m}{2}$ ，即 $num_{min} = \left\lfloor \frac{n+m+1}{2} \right\rfloor$ 。

给出下界情况的一种构造：（下面只讨论 $n \leq m$ 的情况）

$1.$ $m-n$ 为偶数。

先在 $n*n$ 的正方形对角线放置，之后隔一列在任意位置放一个即可。个数为 $\left\lfloor \frac{n+m+1}{2} \right\rfloor$。

$2.$ $m-n$ 为奇数。

先在 $n*n$ 的正方形对角线放置，之后隔一列在任意位置放一个，但如此放置最后一列不会被覆盖，所以还需要在最后一列放置一块。个数为 $\left\lfloor \frac{n+m+1}{2} \right\rfloor$。

# 1004 小z的开箱题解

首先，有一个很显然的性质。由于宝石的幸运值大于0，那么无论从何处开始，选取所有宝石一定是这个起点最优的。但是直接枚举每个起点进行贪心的话，时间复杂度是$O(n^2)$。

注意到一个特殊的数据规模，**a[i]≤10**，由于手中的宝石只会保存质数的幸运值，那么当大于手中宝石的幸运值的下一个质数，与手中宝石相差大于10时，手中宝石的幸运值将不会再增大。所以，答案的上限并不是所有宝石的幸运值之和。

通过查找质数表可以发现，第30个质数(113)，第31个质数为(127)，在此之前，相邻两个质数的差值$\leq$10，所以可能的答案其实只有31种（30个质数以及0）,所以我们转换思路为判断每个起点在这31个数所能选择的最大数为多少。

从0开始，记录相邻两个质数的差值，通过跳质数来判断每个质数是否能到达。预处理出next[i][j]表示第i个位置后幸运值大于等于j的宝石的位置，这样每个起点最多只需要跳30次，反方向进行同样的操作，这样求解的时间复杂度为$O(60n)$。

# 1005 时空碎片收集者题解

## 分析

可以发现"时空跳跃"操作并不会获得额外的碎片,只是会减少时空回溯带来的损失,那么我们可以围绕时空回溯的影响范围进行分析和求解

## 状态定义

由于k的范围很小,我们可以定义
$dp[i][j]$表示来到$i$时空节点 并且 后续还有$j$个节点被时空回溯影响(包括$i$点)

- $j = 0$代表当前没有被回溯影响
- $1 <= j <= k$:节点$i$被回溯影响,且节点$i$后续$j-1$个节点被回溯影响
- $j = k+1$:在节点$i$使用时空回溯,后续$k$个节点被影响

## 状态转移

对于每个节点 $i$，考虑以下转移：

1. 从节点 $i-1$ 直接走来（不跳跃）
   节点 $i$ 不被影响（$j=0$）,从节点 $i-1$ 的 $j=0$ 或 $j=1$ 状态转移而来，加当前节点原始碎片值:
   $dp[i][0] = max(dp[i][0], max(dp[i-1][0], dp[i-1][1]) + a[i])$
2. 节点 $i$ 被影响（$1 \le j \le k$）,从节点 $i-1$ 的 $j+1$ 状态延续而来:
   $dp[i][j] = max(dp[i][j], dp[i-1][j+1] + \lfloor a[i]/2 \rfloor)$
3. 在节点 $i$ 使用时空回溯（只能从不受影响的状态触发）
   $dp[i][k+1] = max(dp[i][k+1], max(dp[i-1][0], dp[i-1][1]) + 2 \times a[i])$
4. 从节点 $i-1$ 使用时空跳跃
   从节点 $i-1$ 的状态 $j$，跳跃到节点 $i+m+1$，收集从 $i$ 到 $i+m$ 的原始碎片（不受回溯影响）:
   $dp[min(i+m, n+1)][max(0ll, j-m-1)] = max(\cdots, dp[i-1][j] + sum(i, min(i+m, n)))$
   其中:
   $sum(l, r) = pre[r] - pre[l-1]$，即原始碎片前缀和

虽然跳到了$i+m+1$节点,但$i+m+1$的选择还未确定,所以这里转移到$i+m$节点,影响回溯的节点比$i-1$少了$m+1$个即$max(0ll, j-m-1)$

最终答案：$\max_{0\le j\le k+1} dp[n+1][j]$，其中 $n+1$ 表示离开状态

## 时间复杂度

状态数：$O(n\times k)$，其中 $k\le 25$，$n$ 总和 $\le 2\times 10^6$
转移复杂度：$O(1)$
总时间复杂度：$O(nk)$，可以通过

# 1006 光辉岁月

约定 $f_i$ 的意义如下：
1：若遗忘第 $i$ 年的约定，$f_i=0$；
2：若想起第 $i$ 年的约定，$f_i$ 表示与下一个能想起的约定的年份与 $i$ 的差值。

那么，对于一段合法区间，必定只包含一个值以及若干 $0$，我们只需要求解出忽略 $0$ 的情况下，最长的只含一个值的序列长度即可。

考虑线段树维护。

区间维护：
最左侧的非零值，以及连续出现次数；
最右侧的非零值，以及连续出现次数；
区间非零值的个数；
区间长度；
区间答案；

区间合并时，考虑左区间的最右侧非零值与右区间的最左侧非零值产生的贡献即可。

对于 $1$ 次区间修改 $[l,r]$：
1：会将 $[l,r-1]$ 范围内的 $f_i$ 均改为 $1$，可以打懒标记维护；
2：（ $l$ 前第一个能想起的约定的年份 ）以及（ 年份 $r$ ）均会受到影响，可以通过线段树二分或者 $std::set$ 求解这两个点的 $f_i$ 值，然后进行两次单点修改。

时间复杂度：$\mathcal{O}(n\log n)$

# 1007 最大三段和

## 题意

给出一个长度为 $n$ 的序列 $a$，要求从中选出三个子段，使得这三个子段的所有数字和最大。每个子段的长度至少为1，并且两个子段之间至少间隔一个数，同时要求第二个子段长度为1。

## 题解

改编自经典题目最大子段和，比较模板，用于筛选出初步了解过动态规划的同学。

先考虑取只一段如何使得数字和最大。设$f_i$表示，以$i$为结尾的所有区间里，和最大为多少。对于第$i$个数，以这个数为末尾的所有区间可以分为两类，一类是以$i-1$为末尾的区间再加上$a_i$，一类就是$a_i$本身。因此转移方程为：

$f[i]=max(f[i-1]+a[i],a[i])$

取一段的答案就是最大的$f_i$，现在考虑取两段。设$g_i$为以$i$为开头的所有区间里，和最大为多少。容易发现$g_i$的求法和$f_i$一样，倒着求即可。枚举中间的断点$i$，答案即为$max f_j+max g_k(j<i<k)$，提前预处理$f$的前缀$max$和$g$的后缀$max$即可。

考虑取三段，因为题目限制第二个子段长度一定为1，因此实际上取三段和取两段其实是一样的。只需把答案改为：$a_i+max f_j+max g_k(j+1<i<k-1)$即可。

# 1008 单十一花钱计划

每一件衣服的价格为 $\min(a+x,b+y)$，即当 $a+x<b+y$ ，即 $b-a<x-y$ 的衣服价格为 $a+x$ , $b-a>=x-y$ 的衣服价格为 $b+y$ ，当 $x-y$ 足够小时所有衣服价格均为 $a+x$，随着 $x-y$ 增大，衣服的价格替换为 $b+y$ 。

先考虑离线时的做法

将询问按照 $x-y$ 从小到大排序，将衣服按照 $b-a$ 从小到大排序，维护取第一种价格的衣服$a_i$的区间最大值，以及第二种价格的衣服的 $b_i$ 区间最大值。使用两棵线段树，处理新增的 $b-a<x-y$ 的衣服，删除 $a_i$，添加 $b_i$，答案为 $\max(\max a+x,\max b+y)$ 。

在线的话我们可以使用可持久化线段树，存储下衣服按照 $b-a$ 排序依次添加的历史线段树版本，在查询时二分出对于 $x-y$ 匹配的版本位置，在对应版本的线段树上查询即可。

时空复杂度均为 $O(nlogn)$ 。

# 1009 临渊羡鱼

显然，答案为数组的 $\max-\min$。

时间复杂度：$\mathcal{O}(n)$

# 1010 收买时间

我们考虑二分。
我们发现，时间越长，可通行的格子越多。也就是说，随着时间的增长，从 $(1,1)$ 走到 $(n,m)$ 会变得越来越可行。

因此我们直接二分答案 $t$。然后我们对于时间 $t$，验证 $(1,1)$ 是否可以到达 $(n,m)$ 即可。

时间复杂度 $O(nlog^2n)$。出题人用了常数比较大的方法，跑了 3.5s。大部分验题人都只跑了 1s。